BrainAI

ProSuite Industrial

API & WRAPPER SUITE

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProKey Integrations-Handbuch**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Release of ProKey Wrapper Documentation
1.1.0	27.02.2026	S. Köhne	Stable	Full API Integration Guide (C#, Java, Node.js, Python, VB.NET)

1 Produktübersicht

Die **ProKey Wrapper-Suite** dient als Hochsprachen-Schnittstelle zur nativen ProKey Hardware-Entropie-Engine. Im Gegensatz zu Standard-Zufallsgeneratoren der jeweiligen Programmiersprachen (wie `java.util.Random` oder `Math.random()`), die lediglich deterministische Pseudozufallszahlen (PRNG) liefern, zapfen diese Wrapper echte physikalische Entropie an.

Dies ist für die Generierung von asymmetrischen Schlüsselpaaren, AES-Masterkeys und hochsicheren Session-Tokens in Enterprise-Umgebungen zwingend erforderlich.

Unterstützte Kernfunktionen:

- **Native Performance:** Direkte Speicherzugriffe ohne unnötige Kopiervorgänge (Zero-Copy-Ansatz).
- **Hardware RNG:** Zugriff auf die optimierten C-Routinen `ProKey_Generate` und `ProKey_Fill256Bit`.
- **Plattformagnostik:** Automatische Erkennung und Bindung der Linux (`.so`) oder Windows (`.dll`) Bibliotheken.

2 Gemeinsame Grundlagen & API-Struktur

Alle Wrapper abstrahieren die native C-Schnittstelle und bieten ein standardisiertes Set an Funktionen und Enumerationen.

Kern-Aufrufe:

1. **`GenerateBytes(int length)`:** Fordert eine beliebige Menge an Entropie an und schreibt diese in ein alloziertes Byte-Array.
2. **`GenerateKey(ProKeyBits bits)`:** Typsichere Generierung anhand der industriellen Standardlängen (128, 256, 512, 1024 Bit).
3. **`Generate256BitKey()`:** Ein Performance-Wrapper, der intern die optimierte native Funktion `ProKey_Fill256Bit` aufruft (Ideal für AES-256).

3 C# (.NET) Integration

Der C#-Wrapper nutzt `P/Invoke` (`DllImport`), um die native C-Bibliothek im unmanaged Speicher anzusprechen.

- **Speichermanagement:** Arrays werden als `byte[]` alloziert und vom Marshaller automatisch an die C-Umgebung übergeben und zurückgeschrieben.
- **Implementierungsbeispiel:**

```
using ProChat.Core.Crypto;

// Generierung eines AES-256 Schlüssels
byte[] masterKey = ProKey.Generate256BitKey();

// Generierung eines 512-Bit Session-Tokens
byte[] sessionToken = ProKey.GenerateKey(ProKeyBits.Bit512);
```

4 Java (JNA) Integration

Die Java-Anbindung erfolgt über **Java Native Access (JNA)**, was den Verzicht auf komplexen JNI-C-Code (Java Native Interface) ermöglicht.

- **Typ-Sicherheit:** Nutzt `NativeSize` für die korrekte plattformübergreifende Behandlung von `size_t` (32-Bit vs. 64-Bit Pointer).
- **Implementierungsbeispiel:**

```
import com.brainai.prokey.ProKey;

// Abruf von 16 Bytes (128 Bit) für einen Initialization Vector (IV)
byte[] iv = ProKey.generateBytes(16);

// Konvertierung in Hex für die Übertragung
String hexToken = ProKey.bytesToHex(ProKey.generateKey(ProKey.ProKeyB
```

5 JavaScript (Node.js) Integration

Verwendet `ffi-napi` und native V8 `Buffer`-Objekte. Dies ist der performanteste Weg, um Datenströme in Node.js-Backends zu verarbeiten, da die Buffer direkt im C++ Speicher der V8-Engine liegen.

- **Einsatzgebiet:** Backend-Services (Express.js/NestJS), JWT-Signatur-Schlüssel.
- **Implementierungsbeispiel:**

```
const { ProKey, ProKeyBits } = require('./prokey');

// Buffer direkt erzeugen und in Hex umwandeln
const aesKeyBuffer = ProKey.generate256BitKey();
const hexKey = aesKeyBuffer.toString('hex').toUpperCase();
```

6 Python Integration

Der Python-Wrapper greift auf das systeminterne `ctypes`-Modul zurück und erfordert keine externen Abhängigkeiten (wie cffi).

- **Sicherheits-Aspekt:** Die Rückgabe erfolgt strikt als unveränderliches `bytes` Objekt, um versehentliche Modifikationen des Schlüssels im Speicher zu verhindern.
- **Implementierungsbeispiel:**

```
from prokey import ProKey, ProKeyBits

# Generiere Nonce
nonce = ProKey.generate_bytes(12)

# 512-Bit Key
token = ProKey.generate_key(ProKeyBits.BIT_512)
```

7 VB.NET Integration

Speziell für Legacy- und OT-Systeme (Operational Technology) konzipiert. Bietet eine typsichere Klasse, die sich nahtlos in ältere Windows-Forms oder Industrie-Steuerungs-Applikationen einbettet.

- **Implementierungsbeispiel:**

```
Imports ProChat.Core.Crypto

Dim myAesKey As Byte() = ProKey.Generate256BitKey()
Console.WriteLine("AES Key: " & ProKey.BytesToHex(myAesKey))
```

8 Best Practices & Sicherheit (Zero-Trust)

- **Garbage Collection (GC) Warnung:** In Hochsprachen (wie Java oder C#) räumt der Garbage Collector den Speicher asynchron auf. Sobald Sie einen Schlüssel nicht mehr benötigen, sollten Sie das Byte-Array nach Möglichkeit manuell mit Nullen überschreiben (`Array.Clear()` in .NET oder `Arrays.fill(key, (byte) 0)` in Java), bevor Sie die Referenz aufheben. Dies minimiert das Angriffsfenster für RAM-Dumps.
- **Hex-Konvertierung:** Übertragen Sie binäre Entropie niemals direkt über Textprotokolle (wie JSON/HTTP). Nutzen Sie immer die in den Wrappern mitgelieferten oder sprachspezifischen Hex- oder Base64-Konvertierungsfunktionen.
- **Hardware-Limitierungen:** Echte Hardware-Entropie ist eine endliche Ressource (physikalische Generierungsrate). Nutzen Sie ProKey für Master-Schlüssel, Salts und Session-Tokens. Für massenhafte, extrem schnelle Datenströme (z.B. Festplatten-Wiping) sollten Sie einen CSPRNG (Cryptographically Secure PRNG) verwenden, der initial mit einem ProKey-Seed gefüttert wurde.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**